
EMPLOYMENT MANAGEMENT SYSTEM

Foundations

Layers, the request lifecycle, the database, and configuration

ARCHITECTURE AND CONFIGURATION • 2 OCTOBER 2026

Contents

01	1. The backend layers	3
02	2. One HTTP request, end to end	4
03	3. Dependency injection, Lombok, and why there are no <code>new</code> keywords	5
04	4. The database schema	6
05	5. Why <code>FetchType.LAZY</code> everywhere	7
06	6. <code>pom.xml</code> - what each dependency is for	8
07	7. <code>application.yaml</code>	8
08	8. What happens on startup	9
09	9. <code>@Transactional</code> in plain language	10
10	10. Pagination and Specifications	11
11	11. Mappers (MapStruct)	11

Everything else in these docs builds on this page. Read it fully before the feature docs.

1. The backend layers

```

HTTP request
|
v
[Servlet container: Tomcat]           Spring Boot embeds Tomcat
|
v
[Security filter chain]               security/config/SecurityConfig.java
                                     → JwtAuthenticationFilter
|
v
[Controller]                         e.g. EmployeeController
                                     @Valid, @RequestBody, @PreAuthorize
|
v
[Service]                           e.g. EmployeeServiceImp
                                     business rules, @Transactional
|
v
[Repository]                        e.g. EmployeeRepository
                                     Spring Data JPA / Specifications
|
v
[Entity]                            e.g. Employee.java (@Entity)
|
v
MySQL table "employees"
  
```

Then the response goes back up the same path:

```

MySQL → Repository → Entity → Mapper → DTO → Controller → JSON → browser
  
```

Why the layers are separate

- **Controller** is the only layer that knows about HTTP. If you change the URL from `/api/employees` to `/api/employees`, you only touch the controller.
- **Service** is the only layer that knows business rules. If the salary rule changes, you only touch the service.
- **Repository** is the only layer that knows SQL/JSQL. If the table is renamed, you only touch `@Table(name = ...)` or the repository.
- **Mapper** is the only layer that knows the difference between a database row and a JSON object.

Interface + implementation split (a deliberate choice)

Every domain has an interface and an implementation:

```

EmployeeService.java    ← interface
  
```

```
EmployeeServiceImp.java ← implementation
```

`Controller` depends on the **interface**, not on `EmployeeServiceImp`:

```
private final EmployeeService employeeService;
```

JAVA

Spring injects `EmployeeServiceImp` because it is the only `@Service` in the package that implements the interface. This split costs an extra file but lets you swap the implementation (e.g. for a mock in tests) without touching controllers.

Note the naming: the implementations end in `Imp`, not `Impl`. That is the project's convention, so `AuthServiceImp`, `UserServiceImp`, `RoleServiceImp`, `DepartmentServiceImp`, `EmployeeServiceImp`, `AttendanceServiceImp`.

2. One HTTP request, end to end

Take the simplest read endpoint: `GET /api/employees/paged?page=0&size=10`.

Step 1 - Browser. `src/api/http.js:32`:

```
export async function fetchEmployees(page) {
  return api(`/api/employees/paged?page=${page}&size=10`);
}
```

JS

`api()` (line 4) reads the token from the Zustand store, adds `Authorization: Bearer <jwt>`, calls `fetch`, throws on non-2xx, and returns JSON.

Step 2 - Servlet. Tomcat receives `GET /api/employees/paged`.

Step 3 - Security filter chain. `SecurityConfig.java:31-53`:

- CSRF is disabled (line 32) - safe because there are no cookies; the token is sent manually in a header, so a browser cannot be tricked into attaching it automatically.
- `.requestMatchers("/api/auth/**", "/error").permitAll()` (line 35) - login, register and the error page are public. Everything else needs authentication.
- `.anyRequest().authenticated()` (line 36).
- Sessions are `STATELESS` (line 39) - the server never stores a session; the token is the entire proof of identity.
- `JwtAuthenticationFilter` runs **before** `UsernamePasswordAuthenticationFilter` (line 53) so the `SecurityContext` is already populated by the time authorization is evaluated.
- If authentication never succeeded, the `authenticationEntryPoint` (line 42) writes `401 {"status":401,"message":"Unauthorized: invalid or missing token"}`.

- If the user is authenticated but lacks the role, the `accessDeniedHandler` (line 47) writes `403` `{"status":403,"message":"Forbidden: insufficient permissions"}`.

Step 4 - Method security. `@EnableMethodSecurity` (line 22) is what makes `@PreAuthorize` annotations work at all. `EmployeeController` is annotated at class level with `hasAnyRole('ADMIN','MANAGER')`, so an `EMPLOYEE` calling this endpoint is rejected with `403` here - before any business logic runs.

Step 5 - Controller. `EmployeeController.getPaged(...)` receives `page` and `size` as `@RequestParam` and delegates to the service.

Step 6 - Service. `EmployeeServiceImp.getPagedEmployees(page, size)` builds a `Pageable` and calls the repository. `JpaRepository.findAll(pageable)` is implemented by Spring Data - no query is written by hand.

Step 7 - Repository / Hibernate. Spring Data generates `select ... from employees limit ?, ?` plus a `count(*)` query, and maps rows to `Employee` objects. Lazy fields (`department`, `attendanceList`) are **not** loaded yet.

Step 8 - Mapper. `EmployeeMapper` (MapStruct) converts each `Employee` to an `EmployeeResponse`. Because it uses `LEFT JOIN FETCH e.department` style fetching, the department is available here; otherwise this would throw a `LazyInitializationException`.

Step 9 - Response. Spring serialises the `Page<EmployeeResponse>` to JSON:

```

{
  "content": [ { "id": 1, "firstName": "Jane", ... } ],
  "pageable": { ... },
  "totalElements": 5,
  "totalPages": 1,
  "size": 10,
  "number": 0
}
```

JSON

Step 10 - Frontend. TanStack Query stores that object under the query key `["employees", 0, ""]` and the table renders `data.content`.

3. Dependency injection, Lombok, and why there are no new keywords

- `@RequiredArgsConstructor` on a class generates a constructor for every `private final` field. That is why controllers/services have no constructor written by hand, and why their dependencies are `final`.
- Spring finds that single constructor and injects the beans it needs. If a bean is missing, the app fails at **startup**, not at request time.
- `@Getter` / `@Setter` / `@Data` / `@AllArgsConstructor` / `@NoArgsConstructor` are Lombok. They remove boilerplate getters/setters. `@Data` on a DTO also generates getters/setters, `equals`, `hashCode`, `toString` and a required-args constructor.

4. The database schema

Four tables plus one join table.

departments

Column	Type	Notes
<code>id</code>	BIGINT	PK, auto increment
<code>name</code>	VARCHAR	unique, not null
<code>description</code>	VARCHAR	not null

`Department.employeeList` is `@OneToMany(mappedBy = "department", LAZY)` - the foreign key lives on the employee side.

employees

Column	Type	Notes
<code>id</code>	BIGINT	PK
<code>first_name</code> , <code>last_name</code>	VARCHAR	
<code>email</code>	VARCHAR	unique, not null
<code>phone</code>	VARCHAR	unique
<code>hire_date</code>	DATE	
<code>salary</code>	DECIMAL	mapped to <code>BigDecimal</code>
<code>department_id</code>	BIGINT	FK -> <code>departments.id</code> , nullable

`Employee.attendanceList` is `@OneToMany(mappedBy = "employee", cascade = ALL, orphanRemoval = true)` - deleting an employee deletes their attendance rows.

attendances

Column	Type	Notes
<code>id</code>	BIGINT	PK
<code>check_in</code>	TIME	null when absent
<code>check_out</code>	TIME	null when absent
<code>date</code>	DATE	not null

Column	Type	Notes
<code>total_hours</code>	DOUBLE	written by the seeder only
<code>status</code>	VARCHAR	enum stored as text (<code>@Enumerated(EnumType.STRING)</code>)
<code>employee_id</code>	BIGINT	FK -> <code>employees.id</code> , not null

`@Table( ... , uniqueConstraints = @UniqueConstraint(columnNames = {"employee_id", "date"}))` means **one attendance row per employee per day**, enforced by the database.

users

Column	Type	Notes
<code>id</code>	BIGINT	PK
<code>username</code>	VARCHAR	unique
<code>password</code>	VARCHAR	BCrypt hash
<code>created_at</code>	DATETIME	<code>@CreationTimestamp</code>
<code>enabled</code>	BOOLEAN	
<code>employee_id</code>	BIGINT	unique FK -> <code>employees.id</code> , nullable

roles and users_roles

`roles(id, name)` holds `ROLE_ADMIN` , `ROLE_MANAGER` , `ROLE_EMPLOYEE` . `users_roles(user_id, role_id)` is the join table for the `@ManyToMany` .

Relationship diagram

```

Department 1 — * Employee 1 — * Attendance
                |
                | (User.employee is OneToOne, unique FK)
                1
User * — * Role

```

TEXT

5. Why FetchType.LAZY everywhere

`LAZY` means "do not load this until someone asks". It is the default choice because loading an employee's full attendance history for a list of 10 employees would be wasteful.

The consequence you must know: **lazy data can only be read inside an open transaction**. Outside it you get `LazyInitializationException` . That is exactly why:

- the mappers (`@Component` MapStruct classes) are the layer that touches lazy getters, and
- `AttendanceServiceImp.searchAttendance` is annotated `@Transactional(readOnly = true)` so `AttendanceMapper` can read `attendance.getEmployee().getFullName()` while the transaction (and therefore the Hibernate session) is still open.

6. pom.xml - what each dependency is for

- `spring-boot-starter-parent:4.0.2` - dependency versions come from here.
- `<java.version>17</java.version>` - the language level.
- `spring-boot-starter-data-jpa` - Hibernate + Spring Data repositories + the `@Entity` machinery.
- `spring-boot-starter-validation` - `@NotBlank`, `@Size`, `@Email`, `@Valid`.
- `spring-boot-starter-webmvc` - `@RestController`, JSON, Tomcat.
- `spring-boot-starter-security` - the filter chain, `AuthenticationManager`, BCrypt.
- `jjwt-api` / `jjwt-impl` / `jjwt-jackson` 0.11.5 - create and parse JWTs. `impl` and `jackson` are runtime-only because your code only compiles against `api`.
- `mysql-connector-j` - the JDBC driver (runtime only).
- `lombok` + `lombok-mapstruct-binding` - **the binding is required**, otherwise MapStruct cannot see Lombok-generated getters and silently produces empty DTOs.
- `mapstruct:1.5.5.Final` - generates `EmployeeMapperImpl` etc. at compile time.
- `spring-boot-configuration-processor` - better IDE hints for `@Value`.
- `spring-boot-starter-validation-test` / `spring-boot-starter-webmvc-test` - the test starters.

In `<build>`, `maven-compiler-plugin` registers **both** annotation processors (Lombok and MapStruct). This is why `mvn clean` is required after adding a mapper - stale generated classes otherwise survive.

7. application.yaml

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/employee_management_system
    username: <dev user>           # committed development credentials - see the
    password: <dev password>       # security notes in section 10
  jpa:
    hibernate:
      ddl-auto: update             # create/update tables from the entities at startup
      show-sql: true               # print every SQL statement
    properties:
      hibernate:
```

```

        format-sql: true           # pretty-print the SQL
    security:
        user:
            name: disabled         # disables Spring Boot's default generated user
            password: disabled

    server:
        port: 8080

    jwt:
        secret: <base64 HMAC secret> # real value lives in application.yml
        expiration: 86400000         # 1 day in milliseconds

```

The real values are redacted above on purpose. The file on disk does contain a committed database password and a committed JWT signing secret, which is exactly the configuration problem described in section 10 - a document that gets read widely is the last place those strings should be repeated. The structure and the explanations below are unaffected by the redaction.

Notes you should be able to explain:

- `ddl-auto: update` means Hibernate **creates the tables for you** from the `@Entity` classes and adds missing columns. It never deletes anything. This is why a brand-new empty MySQL database works with no SQL script.
- `jwt.secret` is a **Base64** string. `JwtService.getSignInKey()` decodes it with `Decoders.BASE64` and rebuilds the HMAC key.
- `jwt.expiration` is milliseconds, and it is added to "now" when the token is created.
- The `spring.security.user` block exists purely to stop Spring Boot from logging in with a default password.
- These values are committed and are **not production safe** - see `08-errors-and-glossary.md`.

8. What happens on startup

1. `EmsApplication.main` calls `SpringApplication.run`.
2. `@SpringBootApplication` triggers component scanning of `com.example.ems`, so every `@Component`, `@Service`, `@Repository` (via JPA), `@RestController` and `@Configuration` class is discovered.
3. Hibernate connects to MySQL, runs the `ddl-auto: update` DDL, and prints SQL.
4. `SecurityConfig` builds the filter chain; `JwtAuthenticationFilter`, `CustomUserDetailsService`, `JwtService`, the `AuthenticationManager` and the `BCryptPasswordEncoder` bean are all created.
5. `DataSeeder.run(...)` executes. See section 9.
6. Tomcat starts listening on 8080.

DataSeeder

`DataSeeder` implements `CommandLineRunner`, so `run(String ...)` is called once at startup, after the context is ready.

```

if (roleRepository.count() == 0) {           // completely fresh database
    seedRolesAndDepartments();               // 3 roles + 3 departments
    seedEmployees(departments);              // 5 employees
    seedAttendance(employees);               // 4 days of check-in data each
    backfillAbsent(employees);               // weekdays without a check-in → ABSENT
    return;
}
if (attendanceRepository.count() == 0) {     // existing DB, empty attendance
    ... seed + backfill ...
} else {
    backfillAbsent(employeeRepository.findAll());
}

```

- `seedRolesAndDepartments()` inserts `ROLE_ADMIN`, `ROLE_MANAGER`, `ROLE_EMPLOYEE` and the departments Engineering / HR / Finance.
- `seedEmployees()` inserts Jane Smith, John Doe, Alex Brown, Maria Garcia, Omar Haddad across those departments.
- `seedAttendance()` inserts, for each employee, the previous 4 days with `checkIn 09:00`, `checkOut 17:30`, `totalHours 8.5`, and status `LATE` for every 3rd employee (index `i % 3 == 0`) else `PRESENT`.
- `backfillAbsent()` walks the previous 14 days, skips Saturday and Sunday, and for any `(employee, date)` pair that has no row yet (`existsByEmployeeIdAndDate`) inserts an `ABSENT` row with null times.

Critical gap: the seeder creates **zero** `User` rows. There is no admin account and no manager account in a fresh database. You must insert one manually (see `08-errors-and-glossary.md`), or use an existing database. Registration always creates `ROLE_EMPLOYEE`.

9. @Transactional in plain language

- `@Transactional` on a class: every public method runs in one database transaction; if one throws, everything it did is rolled back.
- `@Transactional(readonly = true)`: the same, but Hibernate skips dirty checking and flushing, so it is faster - and it keeps the persistence context open long enough for lazy loading.
- You see it on `UserServiceImp`, `AuthServiceImp` (class level), `AuthServiceImp.getCurrentUser` and `AttendanceServiceImp.searchAttendance`.

10. Pagination and Specifications

Pagination is `Pageable` in, `Page` out. Spring Data does the `limit/offset` and the `count` for you. `Page` has the fields the frontend reads: `content`, `totalPages`, `totalElements`, `number`, `size`, `first`, `last`.

Specifications are Spring Data JPA's Criteria-API builder, used only by attendance because only attendance needs dynamic filtering. `AttendanceRepository` extends `JpaSpecificationExecutor<Attendance>`, and `AttendanceSpecification` returns one `Specification` per filter; the repository `AND`s them together. That means the SQL's `WHERE` clause is assembled from exactly the filters that were provided.

11. Mappers (MapStruct)

```
@Mapper(componentModel = "spring")
public interface AttendanceMapper {
    AttendanceResponse toResponse(Attendance attendance);
}
```

JAVA

MapStruct runs at **compile time** and generates `AttendanceMapperImpl`. You never write the implementation; you only declare the method. `componentModel = "spring"` adds `@Component`, so the generated class is a bean that the service can inject.

The mapper is also the natural place to *flatten* data for the frontend, e.g. `AttendanceResponse` contains `employeeName` instead of a nested `Employee` object, so the frontend never triggers lazy loading.